CHARTER

Identify capabilities and areas of potential instability of the “rest api todo list manager”.

Identify documented and undocumented “rest api todo list manager” capabilities.

For each capability, create a script or small program to demonstrate the capability.

Exercise each capability identified with data typical to the intended use of the application.

---

runTodoManagerRestAPI-1.5.5.jar

Area

Exploratory testing for API endpoints

Environment

Windows 11

PC Model

ASUS TUF Gaming F15

START AND END TIME

1:13PM to 2:04PM - 29/09/2026

TESTER

Yessine Chaari - ID: 261179816, E-mail: yessine.chaari@mail.mcgill.ca

TASK BREAKDOWN

#DURATION

51 minutes

The session ran 6 minutes over the 45-minute timebox because we restarted the server to reset data (note 29) and added extra exploratory checks near the end (command injection in note 36, duplicate relationship in note 41, method override in note 42)

SESSION NOTES

---

(1) GET /todos works as expected without any URL parameters. It returns a list of current todos.
(2) GET /todos works as expected with the title as URL parameters. It returns a list of current todos with matching title.
(3) HEAD /todos works as expected. It returns a status 200.
(4) POST /todos with all fields specified works as expected. Creates a todo without an ID with all the fields specified with 201 Created as response code.
(5) We repeat the same POST /todos command and it creates another identical todo but with an ID incremented by one as expected.
(6) GET /todos works as expected with the doneStatus set to false as URL parameters. It returns a list of current todos with doneStatus set to false.
(7) GET /todos works as expected with the doneStatus set to true as URL parameters. It returns an empty list as expected.
(8) We try GET /todos with all specified params and it returns the matching to do as expected. We can also confirm that the URL encoded title was accepted as expected.

(9) For GET /todos with a non specified parameter, it seems to default by returning the list of all todos. The documentation does not specify this behavior, it is something to confirm if it is the desired behavior.

(10) For GET /todos with an empty query parameter returns an empty list of todos as expected.

(11) POST /todos with a missing title returns a 400 error code as expected.

(12) POST /todos with an empty title returns a 400 error code as expected.

(13) POST /todos with malformed JSON returns a 400 error code. The specific expected behavior for this case is not explicitly documented but it is a desirable response.

(14) POST /todos with a specified ID returns a 400 error code as expected. We also note the displayed error message “Invalid Creation: Failed Validation: Not allowed to create with id”

(15) POST /todos with an array as the title, seems to truncate the array and only take the last element and assignt it to the title. For example, we provide a title ["a","c"] and we see in the following GET request that the object was created with title “c”. Is this a desirable behavior?

```
└─$ curl -i -X GET http://localhost:4567/todos
HTTP/1.1 200 OK
Date: Fri, 25 Sep 2026 14:37:31 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"todos":[{"id":"101","title":"c","doneStatus":"false","description":""},{"i
d":"94","title":"Test Todo","doneStatus":"false","description":"Testing POST
"},{"id":"99","title":"123.0","doneStatus":"false","description":""},{"id":"
100","title":"b","doneStatus":"false","description":""},{"id":"98","title":"
123.0","doneStatus":"false","description":""},{"id":"2","title":"XML Respons
e Update","doneStatus":"false","description":""}]}
```

(16) PUT /todos, DELETE /todos, PATCH /todos returns a 405 Method Not Allowed as expected

(17) OPTIONS /todos shows the available endpoint methods as expected.

(18) GET /todos/:id works as expected and returns the corresponding todos with a 200 success code.

(19) GET /todos/:id works as expected when provided with a non existent ID and returns a 404 Not Found with an error message "Could not find an instance with todos/-1"

(20) PUT /todos/:id amends a specific todo and returns a 200success message as expected.

(21) PUT /todos/:id with an empty title returns a 400 Bad Request with error message "Failed Validation: title : can not be empty" as we would expect

(22) PUT /todos/:id with malformed JSON returns a 400 Bad Request with an error message of "java.io.EOFException: End of input at line 1 column 18 path $.". The error message could be more specific.

(23) POST /todos/:id with a title works as expected and returns a 200 success code and amends the title.

(24) DELETE /todos/:id on a valid ID works as expected and returns a 200 success message. Trying to GET /todos/:id on the deleted todo returns 404 Not Found as expected.

(25) OPTIONS /todos/:id shows the available endpoint methods as expected.

(26) PATCH /todos/:id with an invalid ID returns 405 Method Not Allowed as expected.

(27) GET /todos/:id works as expected when provided with a valid ID and expecting an XML response.

(28) PUT /todos/:id works as expected when provided with a valid ID, a JSON body and XML response.

(29) (the server has been restarted to reset data) GET /todos/:id/tasksof works as expected and returns the corresponding project object with a 200 success code.

(30) We create another todo, and use POST /todos/:id/tasksof to add a relationship to project with ID 1. As expected, we receive a 201 Created message. When doing GET /todos/:id/tasksof we observe that the relationship was added in the response object.

(31) With the todo that we just created, we use POST /todos/:id/tasksof again but without specifying any ID in the body. We receive 201 Created which is not really expected, with which project was it linked if there was no ID specified? When doing GET /todos/:id/tasksof we realize that no existing project was linked: the server created a new project (ID 2) with an empty title and linked the todo to it, in addition to project 1 from the previous step.

```
└─$ curl -i http://localhost:4567/todos/3/tasksof
HTTP/1.1 200 OK
Date: Fri, 25 Sep 2026 15:19:21 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"projects":[{"id":"1","title":"Office Work","completed":"false","active":"f
alse","description":"","tasks":[{"id":"2"},{"id":"1"},{"id":"3"}]},{"id":"2"
,"title":"","completed":"false","active":"false","description":"","tasks":[{
"id":"3"}]}]}
```

(32) When doing GET /todos/:id/tasksof with a nonexistent todo ID, we expect 404 Not Found as an answer, however we receive a 200 success message

```
└─$ curl -i http://localhost:4567/todos/999999/tasksof
HTTP/1.1 200 OK
Date: Fri, 25 Sep 2026 15:27:28 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"projects":[{"id":"1","title":"Office Work","completed":"false","active":"f
alse","description":"","tasks":[{"id":"2"},{"id":"1"},{"id":"3"}]},{"id":"2"
,"title":"","completed":"false","active":"false","description":"","tasks":[{
"id":"3"}]},{"id":"1","title":"Office Work","completed":"false","active":"fa
lse","description":"","tasks":[{"id":"2"},{"id":"1"},{"id":"3"}]},{"id":"1",
"title":"Office Work","completed":"false","active":"false","description":"",
"tasks":[{"id":"2"},{"id":"1"},{"id":"3"}]}]}
```

(33) The DELETE /todos/:id/tasksof/:id works as expected on a valid todo ID and valid project ID. It returns a 200 OK success message

(34) The GET and POST /todos/:id/tasksof/:id is expected to return 405 Not Allowed but it rather returns a 404 Not Found which does not match the documentation. PUT and PATCH do return 405 Not Allowed as expected.

```
└─$ curl -i http://localhost:4567/todos/2/tasksof/1
HTTP/1.1 404 Not Found
Date: Fri, 25 Sep 2026 20:18:07 GMT
Content-Type: text/html;charset=utf-8
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)
```

(35) Trying to DELETE /todos/:id/tasksof/:id again the relation that we already deleted does return a 404 Not Found as expected.

(36) A malformed DELETE /todos/:id/tasksof/:id request returns 400 Bad Request as expected (we tried a command injection /todos/1;ls)

(37) The GET /todos/:id/categories works as expected for a valid todo ID with a valid category ID with a 200 success code.

(38) The GET /todos/:id/categories works as expected for a valid todo ID with no categories. It returns a success message 200 with an empty collection

(39) The GET /todos/:id/categories with a non-existing todo ID returns 200 OK success message with a list of existing categories, however we would normally expect 404 Not Found.

```
{"categories":[]}
┌──(yessine㉿YessineASUS)-[/mnt/c/Users/Yessine/Documents/UNI/S7/ECSE_429
oject]
└─$ curl -i http://localhost:4567/todos/999999/categories
HTTP/1.1 200 OK
Date: Fri, 25 Sep 2026 21:15:33 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)
```

(40) We verify that todos #2 does not have a relationship with any category. We use POST /todos/:id/categories to create one with category #1, then we get 201 Created as expected. then we check again with GET /todos/:id/categories and we are able to retrieve it.

(41) We use exactly the same command again for POST /todos/:id/categories to create one with category #1 (even though it already exists). then we check again with GET /todos/:id/categories and we do not have a duplicate relationship as expected.

(42) We get creative and try the PUT /todos/:id/categories method which returns 405 Method Not Allowed as expected. However, we try to test if we can override a PUT method to a POST and we actually get 201 Created. This can create some security risks such as bypassing firewalls or authorizations.

```
yessine@Yessine  yessine@Yessine  yessine@Yessine

└─$ curl -i -X PUT http://localhost:4567/todos/2/categories
HTTP/1.1 405 Method Not Allowed
Date: Tue, 29 Sep 2026 14:36:14 GMT
Content-Type: text/html;charset=utf-8
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)


┌──(yessine㉿YessineASUS)-[/mnt/c/Users/Yessine/Documents/UNI/S7/ECSE_429/Pr
oject]
└─$ curl -i -X PUT \
  http://localhost:4567/todos/2/categories \
  -H "X-HTTP-Method-Override: POST" \
  -H "Content-Type: application/json" \
  -d '{"id":"1"}'
HTTP/1.1 201 Created
Date: Tue, 29 Sep 2026 14:36:16 GMT
```

Program:

- The commands that were ran for tests associated with the number corresponding to their notes

---

1. curl -X GET http://localhost:4567/todos
2. curl -G "http://localhost:4567/todos" --data-urlencode "title=XML Todo"
3. curl -I http://localhost:4567/todos
4. (2 times) curl -X POST http://localhost:4567/todos \
   -H "Content-Type: application/json" \
   -d '{"title": "test task", "doneStatus": true, "description": "a nice and normal description"}'
5. curl -X POST http://localhost:4567/todos \
   -H "Content-Type: application/json" \
   -d '{"title": "test task", "doneStatus": true, "description": "a nice and normal description"}'
6. curl -i “http://localhost:4567/todos?doneStatus=false”
7. curl -i "http://localhost:4567/todos?doneStatus=true"
8. curl -i http://localhost:4567/todos?title=file%20paperwork&doneStatus=false&description=
9. curl -i http://localhost:4567/todos?randomParameter=test
10. curl -i http://localhost:4567/todos?title=
11. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"doneStatus":false,"description":"No title"}'
12. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"title":"","doneStatus":false}'
13. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"title":"Broken Todo"'
14. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"id":"999","title":"Todo With ID"}'
15. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"title":["a","b"]}'
    curl -i -X GET http://localhost:4567/todos
16. curl -i -X PUT http://localhost:4567/todos
17. curl -i -X OPTIONS http://localhost:4567/todos

18. curl -i http://localhost:4567/todos/1
19. curl -i http://localhost:4567/todos/-1
20. curl -i -X PUT http://localhost:4567/todos/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"Updated Todo","doneStatus":true,"description":"Updated description"}'
21. curl -i -X PUT http://localhost:4567/todos/1 \
    -H "Content-Type: application/json" \
    -d '{"title":""}'
22. curl -i -X PUT http://localhost:4567/todos/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"Broken"'
23. curl -i -X POST http://localhost:4567/todos/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"POST Updated Todo"}'
24. curl -i -X DELETE http://localhost:4567/todos/1
    curl -i -X DELETE http://localhost:4567/todos/1
25. curl -i -X OPTIONS http://localhost:4567/todos/1
26. curl -i -X PATCH http://localhost:4567/todos/999999 \
    -H "Content-Type: application/json" \
    -d '{"title":"Should Not Work"}'
27. curl -i http://localhost:4567/todos/1 \
    -H "Accept: application/xml"
28. curl -i -X PUT http://localhost:4567/todos/1 \
    -H "Content-Type: application/json" \
    -H "Accept: application/xml" \
    -d '{"title":"XML Response Update"}'
29. curl -i http://localhost:4567/todos/1/tasksof
30. curl -i -X POST http://localhost:4567/todos \
    -H "Content-Type: application/json" \
    -d '{"title":"Task relationship test"}'
    curl -i -X POST http://localhost:4567/todos/3/tasksof \
    -H "Content-Type: application/json" \
    -d '{"id":"1"}'
    curl -i http://localhost:4567/todos/3/tasksof
31. curl -i -X POST http://localhost:4567/todos/3/tasksof \
    -H "Content-Type: application/json" \
    -d '{}'
    curl -i http://localhost:4567/todos/3/tasksof

32. curl -i http://localhost:4567/todos/999999/tasksof
33. curl -i -X DELETE http://localhost:4567/todos/1/tasksof/1
34. curl -i http://localhost:4567/todos/2/tasksof/1
    curl -i -X POST http://localhost:4567/todos/1/tasksof/1 \ -H "Content-Type: application/json" \ -d '{"id":"1"}'
    curl -i -X PUT http://localhost:4567/todos/1/tasksof/1
    curl -i -X PATCH http://localhost:4567/todos/1/tasksof/1
35. curl -i -X DELETE http://localhost:4567/todos/2/tasksof/1
36. curl -i -X DELETE http://localhost:4567/todos/1%3Bls/tasksof/1
37. curl -i http://localhost:4567/todos/1/categories
38. curl -i http://localhost:4567/todos/2/categories
39. curl -i http://localhost:4567/todos/999999/categories
40. curl -i http://localhost:4567/todos/2/categories
    curl -i -X POST http://localhost:4567/todos/2/categories \
     -H "Content-Type: application/json" \
     -d '{"id":"1"}'
    curl -i http://localhost:4567/todos/2/categories
41. curl -i -X POST http://localhost:4567/todos/2/categories \
     -H "Content-Type: application/json" \
     -d '{"id":"1"}'
    curl -s http://localhost:4567/todos/2/categories
42. curl -i -X PUT http://localhost:4567/todos/2/categories
    curl -i -X DELETE \
     http://localhost:4567/todos/1/categories \
     -H "X-HTTP-Method-Override: POST" \
     -H "Content-Type: application/json" \
     -d '{"id":"1"}'

Summary of session findings

- The core TODO endpoints behave as documented: GET, HEAD, POST, PUT, POST-as-update, DELETE and OPTIONS on /todos and /todos/:id return the expected codes (200, 201, 400, 404, 405) (notes 1-8, 10-14, 16-21, 23-26)
- Filtering GET /todos by title and doneStatus works, including URL-encoded values (notes 2, 6, 7, 8)

- Input validation works for a missing title, an empty title, a client-supplied ID and malformed JSON, all returning 400 (notes 11-14, 21, 22)
- XML responses are available through the Accept header, and a JSON body can be sent with an XML response (notes 27, 28)
- TODO/Project (tasksof) and TODO/Category relationships can be queried, created and deleted, and a repeated category POST does not create a duplicate (notes 29, 30, 33, 35, 37, 38, 40, 41)
- Eight issues were logged (Bugs logged section): undocumented handling of unknown query parameters (9), array title truncation (15), generic malformed-JSON error message (22), blank project created by an empty POST /tasksof (31), 200 instead of 404 for nonexistent todo IDs on /tasksof (32) and /categories (39), 404 instead of 405 on GET/POST /tasksof/:id (34), and the method-override bypass (42)
- Invalid-ID handling is inconsistent: /todos/:id returns 404 for a nonexistent ID (note 19), but the relationship endpoints return 200 with data (notes 32, 39
- The most serious finding is that X-HTTP-Method-Override turns a forbidden PUT into a successful POST (201 Created), which is a security risk (note 42)
- A command-injection style path (/todos/1;ls) was rejected with 400 Bad Request (note 36)

Decisions made

- Started with /todos (GET, filters, HEAD, POST) before moving to /todos/:id and then the relationship endpoints, so the basic behaviour was understood before testing the nested resources that depend on it
- Tested the normal, valid request for each endpoint before its error cases (for example notes 4-8 before 9-14), so that each error could be compared against a known good response
- Repeated the same POST /todos (note 5) to check that IDs auto-increment and that duplicates are allowed
- Restarted the server before the relationship tests (note 29) so that todos and projects were back to a known state after todo 1 had been deleted (note 24)

- Created a fresh todo for the tasksof tests (note 30) instead of reusing existing data, so the relationship results could be attributed to our own requests
- Repeated DELETE on an already-deleted relationship (note 35) and POST on an existing category relationship (note 41) to check how the API handles repeated requests and duplicates.
- Went beyond the documentation with a command-injection style path (note 36) and an X-HTTP-Method-Override header (note 42), because the documented checks were already done and these probe robustness and security
- Used curl for every test so each note maps to one reproducible command in the Program section
- Skipped HEAD on the relationship endpoints, exhaustive malformed-payload validation on nested fields and XML request payloads, because of the 45-minute timebox; they are listed under Areas not covered and Other Session Ideas. The HEAD endpoint is less critical

Files Used and Created

- runTodoManagerRestAPI-1.5.5.jar: the application under test, run locally and reached at http://localhost:4567
- /docs page served by the application (http://localhost:4567/docs): the API documentation used to decide what each endpoint should return
- curl commands: every command run is listed under Program, numbered to match the session notes. These are the script for each capability
- todos_session.sh: This file was created in order to replay the commands that were run during this session in the same order. Note that the server was restarted in step 29 and so the file pauses there in order to ask the user to restart the server and then resume.

Areas not covered

HEAD request for /todos/:id/tasksof

HEAD request for /todos/:id/categories

Exhaustive validation of malformed payloads across all nested relationship fields.

Bugs logged

- Bug #1: For GET /todos with an unspecified or random query parameter, the API defaults to returning the list of all todos; this is undocumented behavior and needs confirmation on whether it is desired.
- Bug #2: POST /todos with an array as the title truncates the array and assigns only the last element to the title (e.g., providing ["a","c"] creates the object with title "c"), which may be undesirable behavior.
- Bug #3: PUT /todos/:id with malformed JSON returns a 400 Bad Request with a generic error message (java.io.EOFException: End of input at line 1 column 18 path $.) that could be more specific.
- Bug #4: POST /todos/:id/tasksof executed without specifying any ID in the request body returns 201 Created and unexpectedly creates a new project with an empty title and links the todo to it (no existing project is linked).
- Bug #5: GET /todos/:id/tasksof with a nonexistent todo ID returns a 200 success message instead of the expected 404 Not Found.
- Bug #6: GET and POST /todos/:id/tasksof/:id return 404 Not Found instead of the documented 405 Method Not Allowed (though PUT and PATCH correctly return 405).
- Bug #7: GET /todos/:id/categories with a non-existing todo ID returns a 200 OK success message with a list of existing categories instead of a 404 Not Found.
- Bug #8: Using an X-HTTP-Method-Override: POST header on a forbidden PUT /todos/:id/categories request successfully bypasses the restriction and returns 201 Created, which poses security risks such as bypassing firewalls or authorizations.

Other Session Ideas

Compare error messages systematically with the Swagger API description.

Test XML input payloads for API endpoints alongside JSON.

Confirm bidirectionality of relationships between todos and projects/categories further.

Test HTTP method spoofing (X-HTTP-Method-Override) across all restricted or forbidden endpoints to uncover security bypass vulnerabilities.

Fuzz standard attributes with arrays, objects, or unusual data types to test how the system handles validation and truncation.

Concerns and Areas of Potential Risk

- **Returning data on invalid requests** (such as querying tasksof or categories with nonexistent IDs) presents a data exposure risk where users can see records they should not access.
- **HTTP method overriding**: HTTP method overriding headers (X-HTTP-Method-Override) successfully bypasses standard method restrictions (e.g., overriding a forbidden PUT to a POST to create categories), which poses security risks regarding authorization and firewall bypasses.
- **Undocumented Query Behavior:** GET /todos with an unspecified or random query parameter defaults to returning the entire list of todos rather than ignoring the parameter or returning an error.
- **Improper Type Coercion / Sanitization:** POST /todos accepts an array as a title, silently truncating it and assigning only the last element to the title (e.g., ["a","c"] becomes "c").
- **Ambiguous Relationship Creation Logic:** Executing POST /todos/:id/tasksof without specifying any ID in the request body returns 201 Created and unexpectedly links the task to all existing projects.

Capabilites

- CRUD operations on TODO Entity in JSON, with XML responses available through the Accept header (notes 1, 2, 4, 5, 6, 7, 8, 18, 20, 23, 24, 27, 28)
- HEAD operation on TODO Entity in JSON (note 3)
- Querying, creating, and deleting TODO/Category and TODO/Project relationships in JSON (notes 29, 30, 33, 35, 37, 38, 40, 41)
- Filtering the TODO list by title, doneStatus and description query parameters (notes 2, 6, 7, 8)

- OPTIONS operation listing the allowed methods on /todos and /todos/:id (notes 17, 25)
- Request method overriding for relationship endpoints (note 42)